

Sakum Lang — Design Specification & Living Doctrine

"सकम्" (Sakam) — Sanskrit for "together / with intent".

Sakum Lang is a Sanskrit-keyword systems language with a self-aware engine, a built-in scientific/quantum core, a binary-hash query engine, and a self-rewriting, self-bootstrapping lifecycle.

This document is the **single source of truth** for what Sakum Lang must and must not do. It is aligned from the original vision and kept non-repeating: every rule appears once. The AI agent and the `self` library read this file to learn and to patch the language.

0. Why this language exists

Existing low-level targets (x86-64, RISC-V, ARM, STM, Raspberry Pi, Arduino) are excellent but **siload**: each needs a different toolchain, none carries scientific notation, security, or learning natively. Sakum Lang proposes a **single core** that:

1. Compiles down to native machine code for multiple ISAs (x64, RISC-V, ARM...).
2. Carries LaTeX/scientific notation and vector/math in the machine core.
3. Has a living "engine" (heart / pulse / nerve) so the runtime is observable.
4. Ships a self-learning memory that records survivability metrics and rewrites its own structure to avoid repeating mistakes.
5. Is quantum-ready: qubit ops exist in the core, even when run as a simulator.
6. Uses a creator-owned hash key for encrypted data transmission.
7. Can only be (re)built by its own code or by machine-level code — never by a foreign high-level dependency.

The base must remain **more capable than raw assembly** for the target domains (AI, vectors, advanced mathematics) while still emitting real assembly.

1. DO — required capabilities

1.1 Core language

- **Hinglish (romanized Sanskrit) keywords only.** Every keyword is typeable ASCII. Devanagari is NOT required and is not part of the language surface. The canonical spelling lives in `SAKUM_HINGLISH.md`.

Pure-English keywords (`let` , `fn` , `if` , `else` , `while` , `for` , `return` , `print` , `class` , `and` , `or` , ...) are intentionally NOT part of the language — the compiler/lexer rejects them. Write Hinglish (e.g. `naam` , `yadi` , `lek` , `kriya` , `vapsa`).

- Static-typed where provable, dynamic where ergonomic; a Curry–Howard friendly type core.
- First-class functions, closures, structs, and vectors.
- **SIMD is a first-class feature:** `vektor` (`वेक्टर`) declares a vector; the compiler emits AVX2 / AVX-512 (x86-64), NEON (ARM64), or RVV (RISC-V) without the programmer writing intrinsics. See §1.11.
- Deterministic arithmetic; explicit overflow/underflow handling.

1.2 Compiler & back ends

- Front end (lexer → parser → AST → IR) shared across all targets.
- Back ends emit **raw machine-level code** — never host-language execution. The canonical implementation lives in `assembly/` as handwritten x86-64 assembly (`.intel_syntax noprefix`):
- `sakum_simd.s` — AVX2 vector add (`vpaddd`), the canonical SIMD demo.
- `sakum_eval.s` — hand-written lexer + recursive-descent parser + evaluator for an embedded Sakum source (the language bootstraps itself).
- `sakum_wasm.s` — emits a spec-valid `.wasm` binary byte-by-byte (verified by `wasm-validate` / `wasmtime` / `node`).
- `sakum_self.s` — the `self` engine at machine level: a code buffer that grows by appending generated instruction bytes (continuous growth).
- `sakum_db.s` — the `sanchay` database engine (`kech` / `vektor` / `anukra` / `sthit` / `asthit` / `grantha`) at machine level.
- `sakum_engine.s` — the kernel + runtime engine (`hriday` / `spand` / `naadi`) at machine level: allocator, pulse scheduler, signal bus, and the syscall hub (ring-3 → ring-0 boundary for I/O/memory/time).
- `sakum_tex.s` — the LaTeX → Sakum transpiler at machine level (no Python/bash); reads LaTeX math from stdin, emits Sakum source to stdout. Handles `\frac` , `\sqrt[n]{}` , powers, subscripts, implicit multiplication, unary minus, parentheses, relations, and `bmatrix` .
- The implementation is **raw machine-level assembly only** — there is no Python, bash, or any other host-language layer in the repo. Build/run with the native toolchain (`gcc` / `as`) and validate portable output with `wasm-validate` / `wasmtime` / `node` . Helper tools that were historically written in Python/bash have been retired and reimplemented in assembly.
- One-pass friendly design to keep the compiler simple and bootstrappable.
- Ahead-of-time compile to binary (`.wasm` / `.s` / SIMD) is the primary path.

1.3 Scientific & math core (built-in, not a library import)

- Native LaTeX rendering of expressions (लेख of a math node prints TeX).
- Vectors, matrices, complex numbers, big integers.
- Trig, hyperbolic, special functions (gamma, Bessel stubs), statistics.
- Direct machine-level conversion of LaTeX symbols to IR ops.

1.4 Quantum core

- Qubit type, Hadamard / Pauli / CNOT / phase gates as first-class ops.
- **Quantum circuits** (assembly/sakum_quantum.s , planned): multi-qubit statevector simulator, gate formulas as unitary matrices, and a binary circuit format (QCB1).
- circuit(n) builds a circuit; gate(c, name, *targets, theta=...) applies H/X/Y/Z/S/T, RX/RX/RZ, and controlled CX/CZ.
- circuit_binary(c) → portable binary (byte-serialized gates + targets + params); circuit_formula(c) → LaTeX unitary; measure_circuit(c) → bits.
- Measurement with collapse semantics; simulator now, real backend later.
- Quantum-ready instruction encoding in the IR.

1.5 Self engine (heart / pulse / nerve)

- हृदय (heart): the runtime allocator + GC heartbeat.
- स्पन्द (pulse): periodic tick driving schedulers and the learning loop.
- नाडी (nerve): event/signal bus connecting components.
- The engine is observable: metrics exported as first-class values.

1.6 Query engine

- Always-on. Every construct is addressable by a **binary hash** (#what).
- Queries are categorized by type and split into per-observation files.
- # notes carry the engine's suggestions in machine-code/binary form.

1.7 Self library (self)

- self.create(...) and self.update(...) for living patching of the language and its own modules.
- Self-diagnostic: bug detection, patch generation, git upload.
- A local **self-updater bot** (tools/sakum_bot.sh , triggered by the native raw-assembly server tools/serve.s via tools/serve.sh) reads learn.md / memory.md , webfetch-checks programming-language updates, writes self-patches to self/patches/ , recompiles the x86-64 assembly/ core, and rolls back on any compile failure (see tools/README.md). Triggered by a timer pulse, a POST /update webhook, or a GET /status poll — all runnable

locally. (The old Python `tools/serve.py` was retired; `serve.s` is its machine-code replacement, per §2 no-host-language doctrine.)

1.12 ब्रम्ह (bramann / गुमन) — the web-crawler activity

- ब्रम्ह (literally "to wander / spider") is the Sakum web-crawler + web-scraper activity. It is built **from scratch in raw x86-64 assembly** (`assembly/sakum_bramann.s`): a hand-written HTTP/1.1 GET client, a from-scratch HTML/byte scraper (extracts `<title>` and `<a href>` links with its own loop, no regex lib), and a **quantum-learn** loop that folds each sphere into a binary hash and records what it researched in `research.md`.
- It also owns a **from-scratch webhook receiver** (`assembly/sakum_webhook.s`): raw assembly `socket/bind/listen/accept`, parses `POST /update`, emits a `webhook.update` nerve signal, and runs a self-update cycle. The bot is kept-alive and silently learning: a local timer (`tools/com.sakum.bot.plist` launchd) or `serve.py --pulse N` runs it continuously, always updating its features. What it improves is logged in `upgrade.md` / `update.md`.
- Spheres: the local `Knowledge/` tree, trusted PL sources (`fetch_updates.sh`), and any webhook target. Every researched sphere becomes a `#what` note in the binary-hash ledger (`query_logs/`).

1.8 Security & cryptography

- Creator-owned encryption key (`सूत्र` / `sutra`) installed per system — **not** a SHA-derived key. The user supplies their own key (env `SAKUM_SUTRA_KEY` or file `sakum_key.txt`). No SHA-256 anywhere in the pipeline.
- Encrypted data transmission over the comms layer using the installed key.
- Post-quantum-safe primitives chosen where available.

1.9 Learning & memory

- Survivability metrics collected after each compile/run.
- Mistake ledger: records failures; the agent rewrites code paths to avoid them.
- Agentic memory: learns new mistakes, recreates advanced assembly in-Sakum.
- Self-rewriting: the language may restructure its own definition later, using only its own code or machine-level code.

1.10 Tooling

- Parser, debugger, interpreter, lightweight VM environment.
- A small autonomous "coding AI" that understands binary-hash queries and acts.

1.11 SIMD / vector core (first-class)

- `vektor` (`वेक्टर`) declares a vector; optional `ank` (`अङ्क`) element type.

- Declaration forms:
- `vektor A = vec(1,2,3);` — literal / expression initialization.
- `vektor ank D[1024];` — sized allocation of a zero vector.
- Element-wise `+` `-` `*` `/` `%` work between two vectors or a vector and a scalar (broadcast). Indexing `D[i]` reads/writes a single lane.
- `simd(n)` returns the target-specific SIMD assembly the compiler would emit for a vector add of length `n` (AVX2 / AVX-512 / NEON / RVV).
- `simd_info()` reports the detected ISA, lanes per instruction, and register width.
- The portable bytecode VM carries vector values natively (VEC / VGET / VSET).
- The source program stays portable; the back end chooses the vector width.

1.13 Database engine (`sanchay`)

- A single machine-level store that unifies **six** primitive data shapes, all addressable by the binary-hash query engine (`#what`) and portable across ISAs/OSes via `platform.inc`. Implemented in `assembly/sakum_db.s`:
- **kech** — in-memory key/value store (Redis / Valkey style), sutra-encrypted at rest, optionally persisted.
- **vektor** — vector index for ANN search (Milvus style); distance computed by the SIMD back end (AVX2/AVX-512/NEON/RVV).
- **anukra** — vectorless classical index (B-tree / inverted) for scalars, strings, and structs.
- **sthit** — **stateful** store: mutable, persisted, addressable by hash; keeps state across pulses (the language's durable memory).
- **asthit** — **stateless** store: pure key->value, no persistence, no mutation of prior state; used for request-scoped / side-effect-free data.
- **grantha** — property graph store with typed edges and `naadi` (nerve) driven traversal.
- All six share the `hriday` (heart) allocator and `sutra` (sutra) crypto, so there is one memory model and one security model. Spec: `spec/spec_db.sak`.

1.14 Systems engineering (`तन्त्र` / `tantra`)

- Algorithm + data structure + system design are written **once** in Sakum and lowered to every target in the matrix, all addressable by `#what` and sharing the `हृदय` allocator + `सूत्र` crypto + `नाडी` bus:
- **ISA**: x86-64 (AVX2/AVX-512), ARM64 (NEON), RISC-V rv64 (RVV).
- **OS**: macOS, Linux, Windows.
- Primitives demonstrated in `assembly/sakum_sys.s` (+ `_arm64`, `_riscv64`): `अन्वेष` (binary search), `सारणी` (open-addressing hash table), `चक्र` (ring buffer / producer-consumer on the nerve bus). The SIR form (`stha` / `chot` / `yoj`) is the portable hub the back ends lower from. Spec: `spec/spec_sys.sak`.

2. DON'T — hard constraints

- Do **not** depend on a foreign high-level runtime to (re)build the core. Bootstrap must reach self-hosting from machine-level code only.
- Do **not** use a foreign host language (Python, bash, Go, Rust, C, ...) anywhere in the project — not for the core, not for libraries, not for the engine, not for the kernel, not for the database, and not for helper tools. Anything the project needs that does not yet exist at machine level **MUST** be created at machine level (raw x86-64 / ARM64 / RISC-V assembly). The old `serve.py`, `gen_sir.sh`, and the `sakum_tex.py` transpiler were retired and reimplemented in assembly for this reason.
- Do **not** silently drop security checks in production (Hoare: life-jacket at sea, not on dry land).
- Do **not** repeat rules in this document — align and fix, never duplicate.
- Do **not** let the AI erase prior doctrine messages; preserve and reconcile.
- Do **not** sacrifice readability for cleverness; syntax stays learnable.
- Do **not** leak the creator hash key; it is installed, never printed in clear.
- Do **not** hardcode a single ISA; back ends are pluggable.
- Do **not** make the language slower than necessary: fast translation and efficient object code are design criteria, not afterthoughts.

3. Architecture (target)

```
assembly/                raw x86-64 machine-level core (no host language)
sakum_simd.s             AVX2 vector add (the canonical SIMD demo)
sakum_eval.s             lexer + recursive-descent parser + evaluator (self-hosted front end)
sakum_wasm.s             byte-by-byte WASM binary emitter (portable output)
sakum_self.s             self engine: code buffer that grows by appending instructions
sakum_bramann.s          ब्रम्ह: from-scratch crawler + scraper + quantum-learn loop
sakum_webhook.s          from-scratch raw-assembly webhook receiver (POST /update)
sakum_lib_time.s         microsecond clock: sakum_now_us/ms, sakum_elapsed_us/ms,
                          sakum_rdtsc (added to support latency measurement)
sakum_lib_dns.s          DNS resolver: sakum_dns_resolve(host) -> IPv4 (added to
                          support hostname-based networking)
sakum_speedtest.s        website speed tester: DNS/TCP-connect/TTFB/total latency
                          + throughput, built from scratch using the two libs above
sakum_adv.s              advanced core: OOP vtable (वर्क), memory safety (हृदय),
                          error explainer (व्याख्या), self-learn bug resolver (स्वाध्याय)
sakum_quantum.s          (planned) QCB1 quantum-circuit binary emitter
sakum_arm.s              (planned) aarch64 NEON back end
sakum_db.s               (planned) सञ्चय: kech/vektor/anukra/grantha unified store
sakum_sys.s               (planned) तन्त्र: binary search + hash table + ring buffer (x86_64)
sakum_sys_arm64.s        (planned) तन्त्र ARM64 NEON port
```

```
sakum_sys_riscv64.s (planned) तन्न RISC-V rv64 + RVV port
...
```

Pipeline (all in assembly):

`source.sak → lexer (asm) → parser (asm) → IR → {wasm | x64/ARM asm | SIMD}`
and the `self` engine grows the code buffer between pulses.

Pipeline:

`source.sak → lexer → parser → AST → ir → {vm | compiler→ISA}`
and in parallel the `engine` runs `pulse` ticks feeding `self` + `agent` + `query\_engine`.

4. Roadmap (phased, self-hosting goal)

1. **Seed** (this repo): raw x86-64 assembly core – lexer, recursive-descent parser, evaluator, SIMD/AVX2, WASM emitter, and the self-growing code buffer.
2. **Bootstrap**: port the core to Sakum itself; compile Sakum-with-Sakum.
3. **Native back ends**: emit real x64 / RISC-V / ARM (NEON) from the IR.
4. **Learning loop**: survivability metrics → mistake ledger → self-rewrite.
5. **Quantum backend**: `sakum\_quantum.s` emitting the QCB1 circuit binary.
6. **Autonomy**: self engine extends the code buffer without a host language.

The implementation is machine-level only; there is no Python (or other host-language) layer. The assembly core is the bootstrap.

5. Keyword glossary

Sakum (Devanagari)	Hinglish spelling	Meaning
--- --- ---		
आरम्भ aarambh	program / main block	
नाम / चर naam / char	declare variable	
क्रिया / सूत्र kriya / sutra	function	
यदि yadi	conditional	
अन्यथा / anyotha	anyatha / anyotha	alternative
यावत् / जबतक yavat / jabtak	loop	
पर्यन्तम् paryantam	counted loop	
प्रत्यागम / वापस	pratyagam / vapsa	return
सत्य satya	boolean true	
असत्य asatya	boolean false	
शून्य shunya	nil	
लेख / मुद्रण lekh / mudran	output	
हृदय hriday	engine allocator	
स्पन्द spand	engine tick	
नाडी naadi	signal bus	
सूत्र sutra	creator encryption key (user-installed)	
वर्ग varg	declare a class (object-oriented, vtable dispatch)	
व्याख्या vyakhya	explain an error code into a human message	
स्वाध्याय svadhyaya	self-learn: log a fault, return an Elixir-style patch note	

```
| संतृप्ति | santripti | saturating arithmetic (overflow-safe) |
| वेक्टर | vektor | declare a SIMD vector |
| अङ्क | ank | integer vector element type |
| परिपथ | paripath | build a quantum circuit |
| द्वार | dvar | apply a quantum gate |
| माप | map | measure a qubit / circuit |
| ब्रम्ह | brahma | web-crawler / web-scraper activity (गुमन: to wander) |
| परीक्षा | pariksha | self-test block (परीक्षा { ... }) |
| और / अथवा | aur / athava | logical and / or |
| लंबाई | lambai | length (len) of a vector/string |
| सञ्चय | sanchay | the unified database engine (store) |
| केच | kech | key/value store (Redis / Valkey style) |
| अनुक्र | anukra | vectorless classical index (B-tree / inverted) |
| ग्रन्थ | grantha | graph store (property graph + typed edges) |
| तन्त्र | tantra | systems engineering: algorithm + data structure + system design |
| अन्वेष | anvesh | search / lookup algorithm (e.g. binary search) |
| सारणी | sarani | hash table / associative data structure |
| चक्र | chakra | ring buffer (system-design queue on the nerve bus) |
```

6. Domain keyword registry (Sanskrit-inspired, by subsystem)

The 148 keywords below are reserved across the **entire** Sakum ecosystem and implemented as machine-level library functions in ``assembly/sakum_lib_domains.s`` (x86-64 / ARM64 / RISC-V) plus the registry in ``assembly/sakum_keywords.s``. They are grouped by domain, not by syntax, and are reserved on every target (Windows / macOS / Linux × x86-64 / ARM64 / RISC-V).

Types & Objects

```
`vastu` object · `rupa` type · `akruti` struct · `samuha` collection · `gan`
group · `kosh` map/dict · `shrunkhala` list · `rekha` array · `bindu` point ·
`jod` tuple · `prakar` variant/enum · `lakshan` trait/interface
```

Functions

```
`ahvaan` invoke · `pravah` pipeline (`|>`) · `sangrah` collect · `vibhaj` split ·
`milan` merge · `parivartan` transform · `anukram` sequence · `punaravartan`
recursion · `pratinidhi` delegate · `vistrit` expand · `sankuchit` reduce
```

Pipe operator (`|>`)

The ``|>`` (pravah/pipe) operator passes the result of the expression on its left as the **first argument** to the function call on its right. It enables readable, left-to-right data transformation pipelines:

Without pipe (deeply nested):

```
lek(guna(jodo(5, 3), 2)) # → prints 16
```

With pipe (flat, left-to-right):

5 |> jodo 3 |> guna 2 |> lek # → prints 16

Multi-line pipe:

data

|> sundar_karo() # clean data

|> badhao() # transform

|> chhapao() # output

""

Semantics:

- `a |> f()` → `f(a)`
- `a |> f(b, c)` → `f(a, b, c)`
- `a |> f() |> g()` → `g(f(a))`
- `pravah` is the Hinglish keyword; `|>` is the operator token.

Precedence: Between comparison and arithmetic (same as Elixir: level 170).

Left-associative.

Restrictions:

- Right side must be a function call or identifier resolving to a function
- Pipe chains are desugared at parse time into nested CALL IR nodes
- PIPE may not appear as the first token of a statement

Concurrency

`sutra` thread · `prakriya` process · `samayojan` sync · `samantar` parallel ·
`sandesh` message · `pravahan` stream · `vahini` channel · `prerak` sender ·
`grahak` receiver · `pratiksha` await · `jagrit` wake · `nidra` sleep

Memory

`smriti` memory · `smritikosh` cache · `aavantana` allocate · `mukti` free ·
`sthaan` address · `suchak` pointer · `sandarbh` reference · `sthir` immutable
 · `chal` mutable · `raksha` protection

Filesystem

`granth` file · `granthagar` directory · `path` · `patan` read · `lekhan`
`write` · `jodan` append · `pratilipi` copy · `sthanantar` move · `naamkaran`
`rename` · `vinash` delete

Networking

jaal network · sampark connect · viyog disconnect · pravesha login ·
nirgam logout · agrah request · uttar response · prasara broadcast ·
graha receive · prasthaan send · dvaara port · marg route

AI

prajna intelligence · buddhi reasoning · chintana inference · smarana
recall · adhigama learning · abhyasa training · nirnaya decision ·
drishti vision · shravana audio · vaka speech · bhasha language ·
manana reflection · kalpana imagination · chetana awareness · sankalpa
planning

Robotics

hasta actuator · netra camera · karna mic · charana locomotion ·
gati movement · disha direction · veg speed · santulan balance ·
spandan sensor event · sparsha touch

Quantum

anu quantum · kana particle · adhisthiti superposition · samyojana
entanglement · taranga wave · kaksha orbital/state · kampan oscillation ·
urja energy · parimana measurement · nirikshaan observe

Compiler

varna token · pada symbol · vakya syntax · artha semantics ·
vishlesha parser · sankalana compile · nirmana build · bandhana link ·
chalana execute · sudhar optimize · pariksha validate · utpadaan
generate

Security

raksha secure · gopana encrypt · vigopana decrypt · pramana authenticate
· adhikara authorize · mudra signature · kunji key · gupta private ·
sarvajanika public · kavacha shield/firewall

Distributed

mandala cluster · ganana compute · vitarana distribute · samanvaya
coordinate · samvedana synchronize · pratinidhi replica · nayaka leader ·
anuyayi follower · matdaana consensus · sthirata consistency

Living System (reserved)

`hriday` Heart (resource mgr) · `manas` Mind (planner) · `buddhi` Reasoning engine · `chetana` Conscious context · `smriti` Long-term memory · `sankalp` Goal/intent · `prerna` Motivation/trigger · `indriya` Sensor interface · `drishti` Vision subsystem · `vak` Speech subsystem · `shravan` Audio subsystem · `sparsh` Touch subsystem · `prana` Runtime lifecycle · `atma` Root runtime identity

This file is read by `self` and `agent`. Editing it is the sanctioned way to change the language's intent. Patches are recorded under `self/patches/`.